

Minggu 06: Distribusi Khusus Diskrit

Armein Z. R. Langi 

armein@itb.ac.id, azr.langi@gmail.com

Institut Teknologi Bandung

March 16, 2026

Kuliah 5

Beberapa soal

11. **Soal:** Tulis skrip Python untuk menghitung mean dan variansi dari distribusi probabilitas diskrit yang diberikan dalam bentuk dictionary.

► **Solusi:**

```
dist = {1: 0.2, 2: 0.5, 3: 0.3} # Value: Prob  
mean = sum(x * p for x, p in dist.items())  
variance = sum((x - mean)**2 * p for x, p in dist.items())  
print(f"Mean: {mean}, Variance: {variance}")
```

Mean: 2.1, Variance: 0.49

12. **Soal:** Gunakan `scipy.stats` untuk menghasilkan 1.000 sampel dari variabel acak kustom dan bandingkan rata-rata sampelnya dengan nilai harapan teoretis.

► **Solusi:**

Beberapa soal

```
from scipy.stats import rv_discrete
xk = [10, 11, 12]; pk = [0.2, 0.5, 0.3]
cust_dist = rv_discrete(name='cust', values=(xk, pk))
samples = cust_dist.rvs(size=1000)
print(f"Sample Mean: {samples.mean()}, Theory Mean: {cust_dist.mean()}")
```

Sample Mean: 11.095, Theory Mean: 11.1

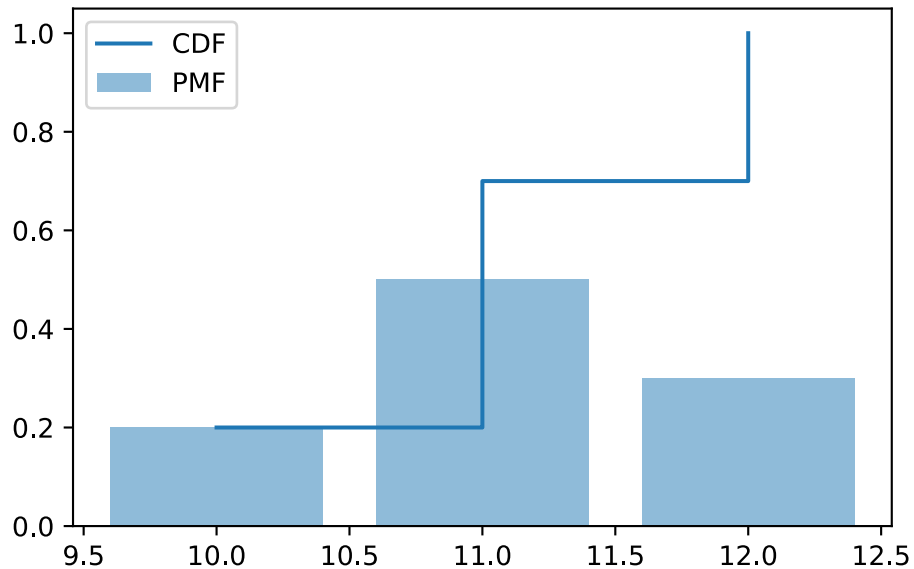
13. **Soal:** Plot fungsi massa probabilitas (PMF) dan CDF dari sebuah variabel acak diskrit menggunakan step plot di matplotlib.

► **Solusi:**

```
import matplotlib.pyplot as plt
import numpy as np
x = [10, 11, 12]; p = [0.2, 0.5, 0.3]
```

Beberapa soal

```
cdf = np.cumsum(p)
plt.step(x, cdf, where='post', label='CDF')
plt.bar(x, p, label='PMF', alpha=0.5)
plt.legend(); plt.show()
```



Beberapa soal

14. **Soal:** Implementasikan fungsi untuk menghitung variansi menggunakan rumus $E[X^2] - (E[X])^2$.

► **Solusi:**

```
def calc_variance(values, probs):  
    ex = sum(v * p for v, p in zip(values, probs))  
    ex2 = sum((v**2) * p for v, p in zip(values, probs))  
    return ex2 - ex**2
```

15. **Soal:** Buat simulasi pelemparan dadu 10.000 kali dan tunjukkan secara visual bagaimana rata-rata kumulatifnya konvergen ke 3,5.

► **Solusi:**

```
import numpy as np; import matplotlib.pyplot as plt  
n = 100
```

Beberapa soal

```
rolls = np.random.randint(1, 7, n)
print(rolls, "\n")
avgs = np.cumsum(rolls) / np.arange(1, n+1)
# plt.plot(avgs); plt.axhline(3.5, color='r'); # plt.show()
```

```
[2 1 4 3 6 6 3 5 2 6 1 6 5 2 4 3 4 6 2 2 4 6 2 2 1 3 4 2 2 3 5 2 2 2 2 1 6
 5 6 6 3 3 3 1 1 5 5 4 3 6 6 4 1 5 2 1 6 6 6 3 6 2 4 3 3 1 1 6 1 1 4 4 2 2
 1 4 5 2 3 2 1 5 3 1 6 2 5 1 4 5 3 6 3 4 3 4 5 6 6 2]
```

Persiapan

Blok 1: Persiapan Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.stats as stats
from scipy.stats import binom, poisson

print("Siap untuk komputasi Minggu 06!")
```

Siap untuk komputasi Minggu 06!

Blok 2: Visualisasi Distribusi Binomial (“Visualizing Luck”)

Kode ini memplot probabilitas jumlah “sukses” (k) dari bereksperimen sebanyak 20 kali ($n = 20$) dengan peluang sukses 50% ($p = 0.5$).

```
# Plot probabilitas sukses 0 hingga 20 dalam eksperimen n=20 dan p=0.5
x = range(21)
plt.bar(x, binom.pmf(x, 20, 0.5))
plt.title("Distribusi Binomial (n=20, p=0.5)")
plt.xlabel("Jumlah Sukses")
plt.ylabel("Probabilitas")
plt.show()
```

Blok 3: Visualisasi Perubahan Bentuk Kurva Binomial

Melihat bagaimana distribusi bergeser seiring dengan berubahnya nilai probabilitas sukses (p).

```
n = 20
for p in [0.1, 0.5, 0.9]:
    plt.plot(binom.pmf(range(n+1), n, p), label=f'p={p}')
plt.legend()
plt.title("Perubahan Bentuk Distribusi Binomial")
plt.show()
```

Blok 4: Membuktikan Hukum Kejadian Jarang (Binomial vs Poisson)

Skenario Inspeksi pabrik: ada 1% kemungkinan produk cacat ($p = 0.01$). Jika kita mengambil sampel 100 unit ($n = 100$), berapa peluang tepat menemukan 2 barang cacat? Mari kita bandingkan perhitungan akurat Binomial vs pendekatan Poisson.

```
n, p = 100, 0.01
```

```
mu = n * p # Rumus hukum kejadian jarang: lambda (mu) = n * p
```

```
print(f"Pendekatan Binomial P(X=2): {binom.pmf(2, n, p)}")
```

```
print(f"Pendekatan Poisson P(X=2): {poisson.pmf(2, mu)}")
```

```
# Hasilnya akan sangat mirip, membuktikan bahwa Poisson bisa menaksir Binomial dengan baik jika n besar dan p kecil.
```

Blok 5: Simulasi Trafik Jaringan (“The Traffic Controller”)

Mensimulasikan kedatangan lalu lintas jaringan menggunakan pemodelan Poisson untuk memonitor trafik normal dan memprediksi lonjakan (*event*).

```
# Mensimulasikan data trafik jaringan selama 1000 jam, dengan rata-rata 5 event/jam
simulasi_trafik = np.random.poisson(lam=5, size=1000)

print(f"Rata-rata trafik hasil simulasi: {np.mean(simulasi_trafik)}")

# Menghitung peluang server mengalami LEBIH DARI 10 event dalam 1 jam (indikasi anomali)
# Menggunakan Cumulative Distribution Function (CDF)
prob_overload = 1 - poisson.cdf(10, 5)
print(f"Probabilitas trafik melebihi 10 event/jam: {prob_overload:.5f}")
```

Antrian

```
import numpy as np
```

```
arrival_rate = 2
```

```
service_rate = 3
```

```
n = 1000
```

```
interarrival = np.random.exponential(1/arrival_rate, n)
```

```
service_time = np.random.exponential(1/service_rate, n)
```

```
arrival_time = np.cumsum(interarrival)
```

```
start_service = np.zeros(n)
```

```
finish_service = np.zeros(n)
```

Antrian

```
for i in range(n):  
    if i == 0:  
        start_service[i] = arrival_time[i]  
    else:  
        start_service[i] = max(arrival_time[i], finish_service[i-1])  
  
    finish_service[i] = start_service[i] + service_time[i]  
  
waiting_time = start_service - arrival_time  
  
print("Average waiting time:", np.mean(waiting_time))
```

Average waiting time: 0.5438222290050063